DATA620 Week 10

April 27, 2026

We start with the data set provided from UCI Machine Learning Repository: Spambase Data Set. We can see that the data provided gives the frequency for 48 different words, 6 different characters (mostly symbols), the length of continuous capital letters in 3 different forms, and finally the classification of spam/ham. Each entry is an individual email and the frequency is how many times these appear in the email.

[1]:
```
import numpy as np
import pandas as pd
# import seaborn as sns
# import nltk
# import random
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score,␣
↪recall_score, f1_score
```

Here we import the data and assign it to a `pandas` dataframe and ensure it is labeled.

[2]:
```
names = pd.read_csv("spambase1.names", header = None)
data = pd.read_csv("spambase.data", delimiter=',', header = None, names =␣
↪names[0])

data.head(5).T
```

[2]:

	0	1	2	3	4
word_freq_make	0.000	0.210	0.060	0.000	0.000
word_freq_address	0.640	0.280	0.000	0.000	0.000
word_freq_all	0.640	0.500	0.710	0.000	0.000
word_freq_3d	0.000	0.000	0.000	0.000	0.000
word_freq_our	0.320	0.140	1.230	0.630	0.630
word_freq_over	0.000	0.280	0.190	0.000	0.000
word_freq_remove	0.000	0.210	0.190	0.310	0.310
word_freq_internet	0.000	0.070	0.120	0.630	0.630
word_freq_order	0.000	0.000	0.640	0.310	0.310
word_freq_mail	0.000	0.940	0.250	0.630	0.630
word_freq_receive	0.000	0.210	0.380	0.310	0.310

word_freq_will	0.640	0.790	0.450	0.310	0.310
word_freq_people	0.000	0.650	0.120	0.310	0.310
word_freq_report	0.000	0.210	0.000	0.000	0.000
word_freq_addresses	0.000	0.140	1.750	0.000	0.000
word_freq_free	0.320	0.140	0.060	0.310	0.310
word_freq_business	0.000	0.070	0.060	0.000	0.000
word_freq_email	1.290	0.280	1.030	0.000	0.000
word_freq_you	1.930	3.470	1.360	3.180	3.180
word_freq_credit	0.000	0.000	0.320	0.000	0.000
word_freq_your	0.960	1.590	0.510	0.310	0.310
word_freq_font	0.000	0.000	0.000	0.000	0.000
word_freq_000	0.000	0.430	1.160	0.000	0.000
word_freq_money	0.000	0.430	0.060	0.000	0.000
word_freq_hp	0.000	0.000	0.000	0.000	0.000
word_freq_hpl	0.000	0.000	0.000	0.000	0.000
word_freq_george	0.000	0.000	0.000	0.000	0.000
word_freq_650	0.000	0.000	0.000	0.000	0.000
word_freq_lab	0.000	0.000	0.000	0.000	0.000
word_freq_labs	0.000	0.000	0.000	0.000	0.000
word_freq_telnet	0.000	0.000	0.000	0.000	0.000
word_freq_857	0.000	0.000	0.000	0.000	0.000
word_freq_data	0.000	0.000	0.000	0.000	0.000
word_freq_415	0.000	0.000	0.000	0.000	0.000
word_freq_85	0.000	0.000	0.000	0.000	0.000
word_freq_technology	0.000	0.000	0.000	0.000	0.000
word_freq_1999	0.000	0.070	0.000	0.000	0.000
word_freq_parts	0.000	0.000	0.000	0.000	0.000
word_freq_pm	0.000	0.000	0.000	0.000	0.000
word_freq_direct	0.000	0.000	0.060	0.000	0.000
word_freq_cs	0.000	0.000	0.000	0.000	0.000
word_freq_meeting	0.000	0.000	0.000	0.000	0.000
word_freq_original	0.000	0.000	0.120	0.000	0.000
word_freq_project	0.000	0.000	0.000	0.000	0.000
word_freq_re	0.000	0.000	0.060	0.000	0.000
word_freq_edu	0.000	0.000	0.060	0.000	0.000
word_freq_table	0.000	0.000	0.000	0.000	0.000
word_freq_conference	0.000	0.000	0.000	0.000	0.000
char_freq_;	0.000	0.000	0.010	0.000	0.000
char_freq_(	0.000	0.132	0.143	0.137	0.135
char_freq_[	0.000	0.000	0.000	0.000	0.000
char_freq_!	0.778	0.372	0.276	0.137	0.135
char_freq_$	0.000	0.180	0.184	0.000	0.000
char_freq_#	0.000	0.048	0.010	0.000	0.000
capital_run_length_average	3.756	5.114	9.821	3.537	3.537
capital_run_length_longest	61.000	101.000	485.000	40.000	40.000
capital_run_length_total	278.000	1028.000	2259.000	191.000	191.000
spam	1.000	1.000	1.000	1.000	1.000

To get a better idea of the impact of individual predictors on the classification of spam and ham by grouping by spam and ham and taking the average value for each predictor.

[3]:
```
mean_data = data.groupby(by='spam').agg('mean').T.sort_values(1, ascending =␣
↪False)
mean_data
```

[3]:

spam	0	1
capital_run_length_total	161.470947	470.619415
capital_run_length_longest	18.214491	104.393271
capital_run_length_average	2.377301	9.519165
word_freq_you	1.270341	2.264539
word_freq_your	0.438702	1.380370
word_freq_will	0.536324	0.549972
word_freq_free	0.073587	0.518362
word_freq_our	0.181040	0.513955
char_freq_!	0.109984	0.513713
word_freq_all	0.200581	0.403795
word_freq_mail	0.167170	0.350507
word_freq_email	0.097292	0.319228
word_freq_business	0.048346	0.287507
word_freq_remove	0.009383	0.275405
word_freq_000	0.007088	0.247055
word_freq_font	0.045226	0.238036
word_freq_money	0.017138	0.212879
word_freq_internet	0.038415	0.208141
word_freq_credit	0.007579	0.205521
word_freq_over	0.044544	0.174876
char_freq_$	0.011648	0.174478
word_freq_order	0.038049	0.170061
word_freq_3d	0.000886	0.164672
word_freq_address	0.244466	0.164650
word_freq_make	0.073479	0.152339
word_freq_people	0.061664	0.143547
word_freq_re	0.415760	0.125091
word_freq_receive	0.021711	0.118434
word_freq_addresses	0.008318	0.112079
char_freq_(	0.158578	0.108970
word_freq_report	0.042403	0.083574
char_freq_#	0.021713	0.078877
word_freq_1999	0.197744	0.043469
word_freq_direct	0.083117	0.036718
word_freq_technology	0.141671	0.029515
char_freq_;	0.050281	0.020573
word_freq_650	0.193806	0.018798
word_freq_hp	0.895473	0.017479
word_freq_edu	0.287184	0.014727

```
word_freq_data                0.150986    0.014562
word_freq_pm                  0.121679    0.012427
word_freq_hpl                 0.431994    0.009173
word_freq_original            0.070581    0.008450
char_freq_[                   0.022684    0.008199
word_freq_85                  0.169455    0.006928
word_freq_project             0.126636    0.006244
word_freq_labs                0.165854    0.005968
word_freq_parts               0.018723    0.004710
word_freq_meeting             0.216808    0.002443
word_freq_conference          0.051227    0.002101
word_freq_415                 0.077787    0.001776
word_freq_george              1.265265    0.001550
word_freq_telnet              0.106033    0.001274
word_freq_table               0.008192    0.001219
word_freq_lab                 0.162794    0.000684
word_freq_857                 0.077306    0.000518
word_freq_cs                  0.072027    0.000055
```

We also want to try and see if the median works better than the mean but we can see a lot of the medians are zero which do not provide a lot of benefit for this analysis.

[4]:
```
median_data = data.groupby(by='spam').agg('median').T.sort_values(1, ascending␣
↪= False)
median_data
```

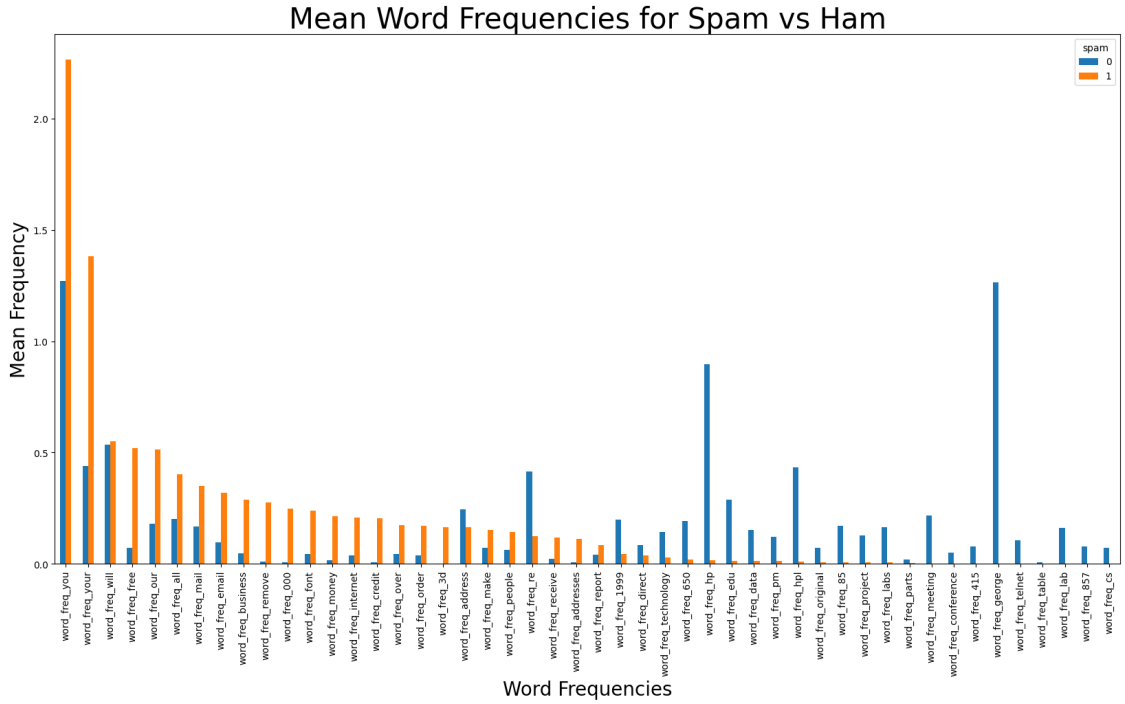
[4]:
```
spam                            0        1
capital_run_length_total    54.0000  194.000
capital_run_length_longest  10.0000   38.000
capital_run_length_average   1.8570    3.621
word_freq_you                0.5100    2.180
word_freq_your               0.0000    1.190
word_freq_will               0.0000    0.430
char_freq_!                  0.0000    0.331
word_freq_all                0.0000    0.300
word_freq_our                0.0000    0.290
word_freq_free               0.0000    0.140
char_freq_$                  0.0000    0.080
char_freq_(                  0.0645    0.065
word_freq_original           0.0000    0.000
word_freq_85                 0.0000    0.000
word_freq_technology         0.0000    0.000
word_freq_1999               0.0000    0.000
word_freq_parts              0.0000    0.000
word_freq_pm                 0.0000    0.000
word_freq_direct             0.0000    0.000
word_freq_cs                 0.0000    0.000
word_freq_meeting            0.0000    0.000
```

```
char_freq_#                  0.0000    0.000
char_freq_[                  0.0000    0.000
word_freq_project            0.0000    0.000
word_freq_re                 0.0000    0.000
word_freq_edu                0.0000    0.000
word_freq_data               0.0000    0.000
word_freq_table              0.0000    0.000
word_freq_conference         0.0000    0.000
char_freq_;                  0.0000    0.000
word_freq_415                0.0000    0.000
word_freq_make               0.0000    0.000
word_freq_857                0.0000    0.000
word_freq_business           0.0000    0.000
word_freq_3d                 0.0000    0.000
word_freq_over               0.0000    0.000
word_freq_remove             0.0000    0.000
word_freq_internet           0.0000    0.000
word_freq_order              0.0000    0.000
word_freq_mail               0.0000    0.000
word_freq_receive            0.0000    0.000
word_freq_people             0.0000    0.000
word_freq_report             0.0000    0.000
word_freq_addresses          0.0000    0.000
word_freq_email              0.0000    0.000
word_freq_telnet             0.0000    0.000
word_freq_credit             0.0000    0.000
word_freq_font               0.0000    0.000
word_freq_000                0.0000    0.000
word_freq_money              0.0000    0.000
word_freq_hp                 0.0000    0.000
word_freq_hpl                0.0000    0.000
word_freq_george             0.0000    0.000
word_freq_650                0.0000    0.000
word_freq_address            0.0000    0.000
word_freq_labs               0.0000    0.000
word_freq_lab                0.0000    0.000
```

To see the frequencies of words most highly associated with spam over ham we can create a bar graph of all the mean values prioritizing the sorting of the spam since that what we should focus on.

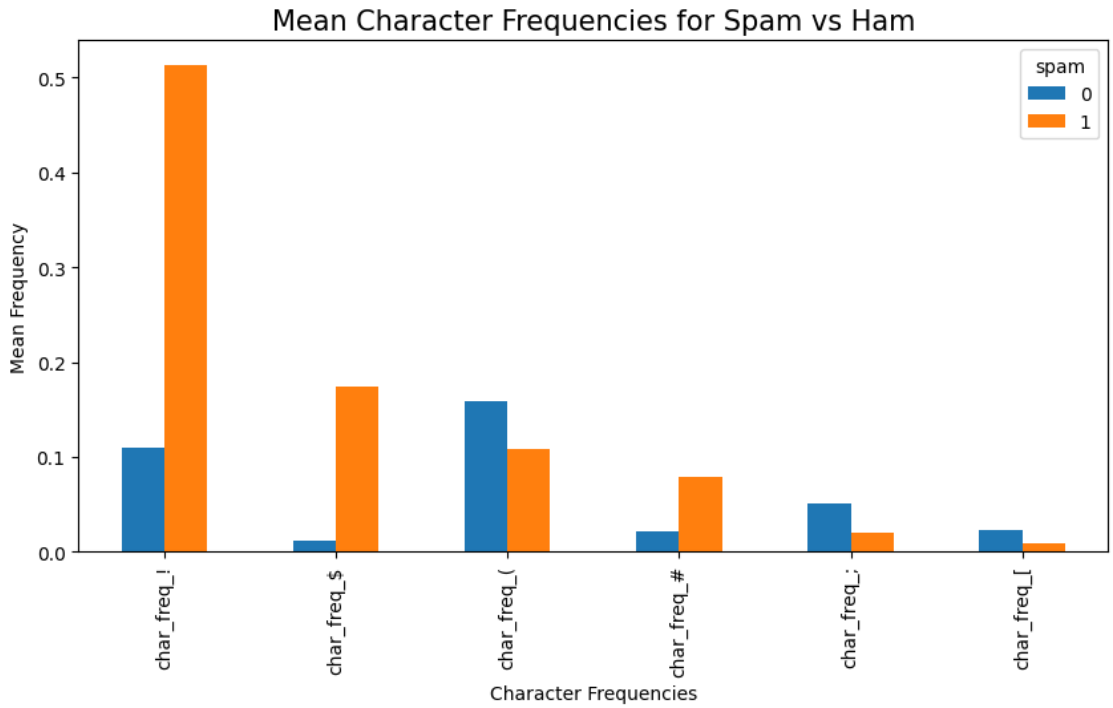
[5]:
```
fig, ax = plt.subplots(figsize = (20,10))
plt.title('Mean Word Frequencies for Spam vs Ham' , size = 30)
plt.xlabel('Word Frequencies', size = 20)
plt.ylabel('Mean Frequency', size = 20)

mean_data.T.filter(like = 'word_').T.plot.bar(ax=ax)
```

```
plt.show()
```



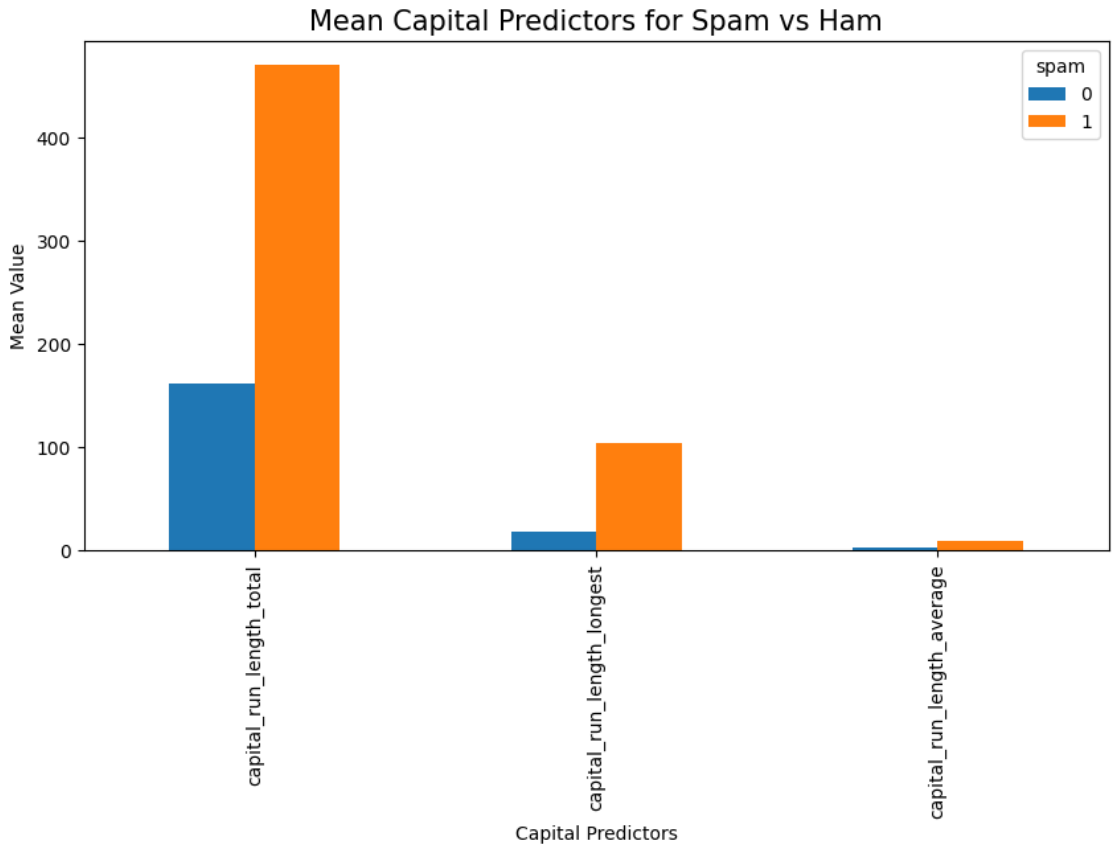
Next we can plot the frequencies of the characters since their distribution is expected to be different from the words, the spam numbers are also prioritized here.

[6]:
```
fig, ax = plt.subplots(figsize = (10,5))
plt.title('Mean Character Frequencies for Spam vs Ham' , size = 15)
plt.xlabel('Character Frequencies', size = 10)
plt.ylabel('Mean Frequency', size = 10)

mean_data.T.filter(like = 'char_').T.plot.bar(ax=ax)
plt.show()
```



Finally we make the same graph for the capital predictors to see which is most influential.

[7]:
```
fig, ax = plt.subplots(figsize = (10,5))
plt.title('Mean Capital Predictors for Spam vs Ham' , size = 15)
plt.xlabel('Capital Predictors', size = 10)
plt.ylabel('Mean Value', size = 10)

mean_data.T.filter(like = 'capital_').T.plot.bar(ax=ax)
plt.show()
```

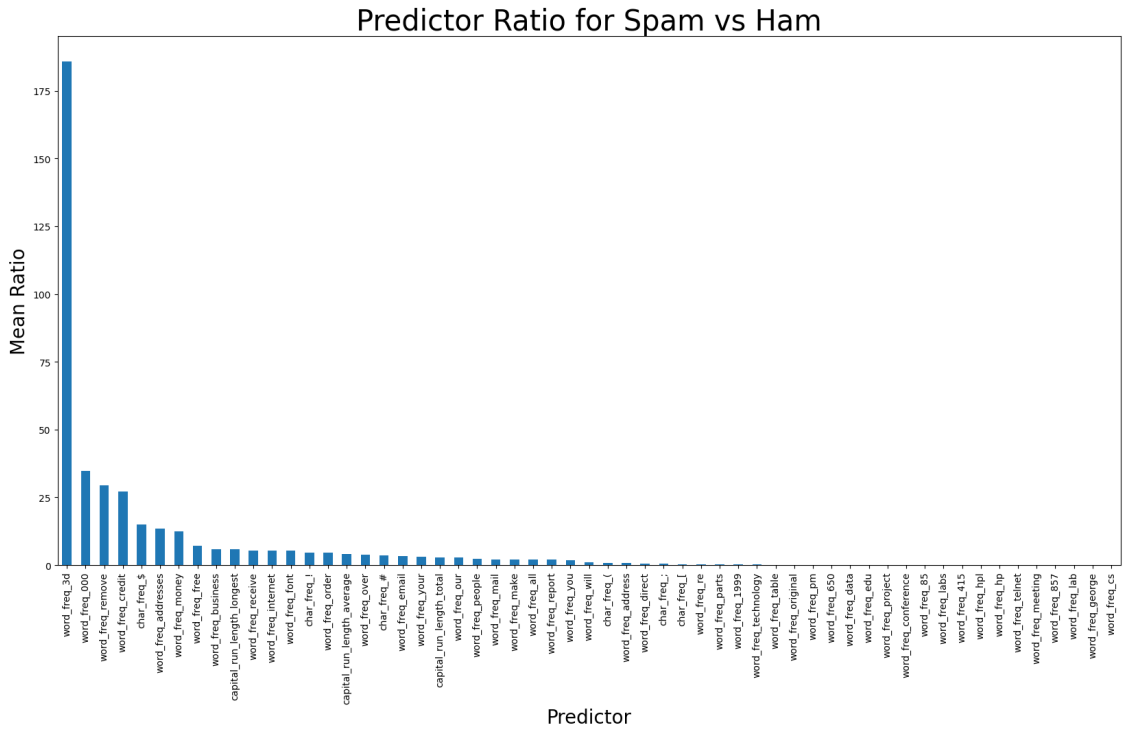


These distributions can show us which predictors may be good for telling us which emails are spam or ham. For example, `word_freq_you`, `char_freq_!`, `capital_run_length_total` have pretty high averages for spam over ham so they will likely be good predictors. But just because they have high averages doesn't mean that relative to the ham it is easier to use as a predictor.

[8]:
```
mean_data['ratio'] = mean_data[1]/mean_data[0]
mean_data = mean_data.sort_values('ratio', ascending = False)
```

To show which predictors have higher averages for spam relative to ham we can take the ratio of the two values and then make a graph of those values.

[9]:
```
fig, ax = plt.subplots(figsize = (20,10))
plt.title('Predictor Ratio for Spam vs Ham' , size = 30)
plt.xlabel('Predictor', size = 20)
plt.ylabel('Mean Ratio', size = 20)

mean_data['ratio'].plot.bar(ax=ax)
plt.show()
```



We can define a function to create individual random forest models to see which is most impactful given the number of predictors.

[10]:
```
def randomForest(X,y):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,␣
 ↪random_state=314)

    rf = RandomForestClassifier(n_estimators=100, random_state=314)
    rf.fit(X_train, y_train)
    y_pred = rf.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    # print('Accuracy score: {:.2f}%'.format(acc * 100))

    return acc, y_pred

y = data['spam'].values
```

Here we can see that the number of predictors definitely has an impact of the accuracy score of the model with marginal returns the more predictors that get added.

[11]:
```
cols = data.columns

acc_lst = []
```

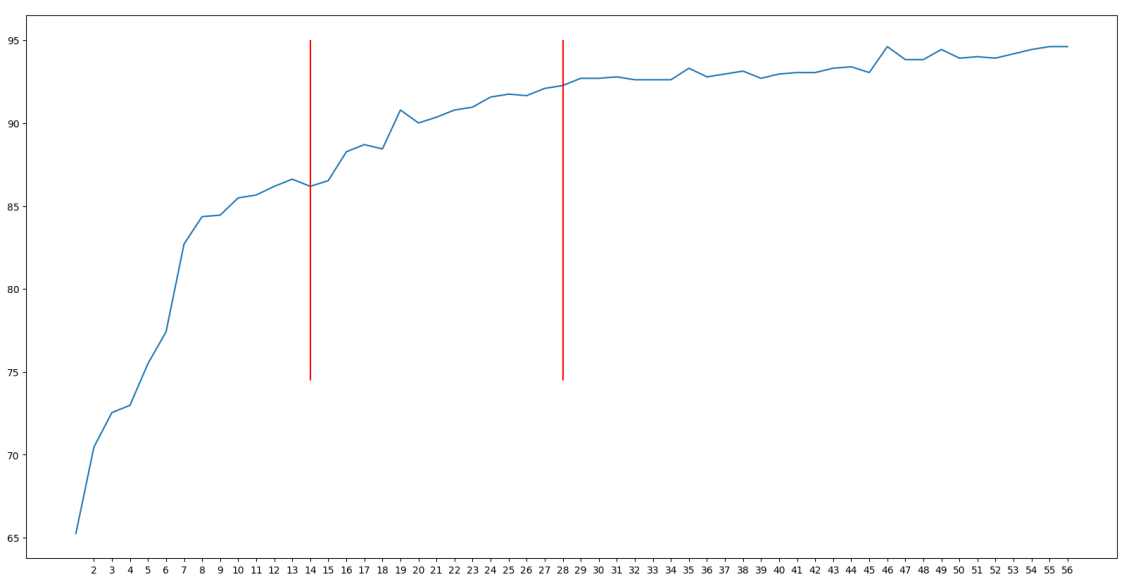
```python
for i in range(1,57):
    X = data[cols[:i]].values
    acc, y_pred = randomForest(X,y)
    acc_lst.append([i,acc*100])

for ea in acc_lst:
    print('Feature set {}'.format(ea[0]) + ' accuracy score: {:.2f}%'.
 ↪format(ea[1]))
```

```
Feature set 1 accuracy score: 65.25%
Feature set 2 accuracy score: 70.46%
Feature set 3 accuracy score: 72.55%
Feature set 4 accuracy score: 72.98%
Feature set 5 accuracy score: 75.50%
Feature set 6 accuracy score: 77.41%
Feature set 7 accuracy score: 82.71%
Feature set 8 accuracy score: 84.36%
Feature set 9 accuracy score: 84.45%
Feature set 10 accuracy score: 85.49%
Feature set 11 accuracy score: 85.66%
Feature set 12 accuracy score: 86.19%
Feature set 13 accuracy score: 86.62%
Feature set 14 accuracy score: 86.19%
Feature set 15 accuracy score: 86.53%
Feature set 16 accuracy score: 88.27%
Feature set 17 accuracy score: 88.71%
Feature set 18 accuracy score: 88.44%
Feature set 19 accuracy score: 90.79%
Feature set 20 accuracy score: 90.01%
Feature set 21 accuracy score: 90.36%
Feature set 22 accuracy score: 90.79%
Feature set 23 accuracy score: 90.96%
Feature set 24 accuracy score: 91.57%
Feature set 25 accuracy score: 91.75%
Feature set 26 accuracy score: 91.66%
Feature set 27 accuracy score: 92.09%
Feature set 28 accuracy score: 92.27%
Feature set 29 accuracy score: 92.70%
Feature set 30 accuracy score: 92.70%
Feature set 31 accuracy score: 92.79%
Feature set 32 accuracy score: 92.62%
Feature set 33 accuracy score: 92.62%
Feature set 34 accuracy score: 92.62%
Feature set 35 accuracy score: 93.31%
Feature set 36 accuracy score: 92.79%
Feature set 37 accuracy score: 92.96%
Feature set 38 accuracy score: 93.14%
Feature set 39 accuracy score: 92.70%
```

```
Feature set 40 accuracy score: 92.96%
Feature set 41 accuracy score: 93.05%
Feature set 42 accuracy score: 93.05%
Feature set 43 accuracy score: 93.31%
Feature set 44 accuracy score: 93.40%
Feature set 45 accuracy score: 93.05%
Feature set 46 accuracy score: 94.61%
Feature set 47 accuracy score: 93.83%
Feature set 48 accuracy score: 93.83%
Feature set 49 accuracy score: 94.44%
Feature set 50 accuracy score: 93.92%
Feature set 51 accuracy score: 94.01%
Feature set 52 accuracy score: 93.92%
Feature set 53 accuracy score: 94.18%
Feature set 54 accuracy score: 94.44%
Feature set 55 accuracy score: 94.61%
Feature set 56 accuracy score: 94.61%
```

We can see by graphing the accuracy scores for the number of predictors. The 14 and 28 predictor models have the some of the highest accuracy scores relative to the other number of predictors around it. More predictors either marginally improves the accuracy score or even makes it worse.

[12]:
```
acc_df = pd.DataFrame(acc_lst, columns = ['feature', 'accuracy'])

plt.figure(figsize = (20,10))
plt.plot(acc_df['feature'], acc_df['accuracy'])
plt.vlines(x=[14,28], ymin = 74.5, ymax = 95, color = 'red')
plt.xticks(np.arange(2, 57, 1))
plt.show()
```

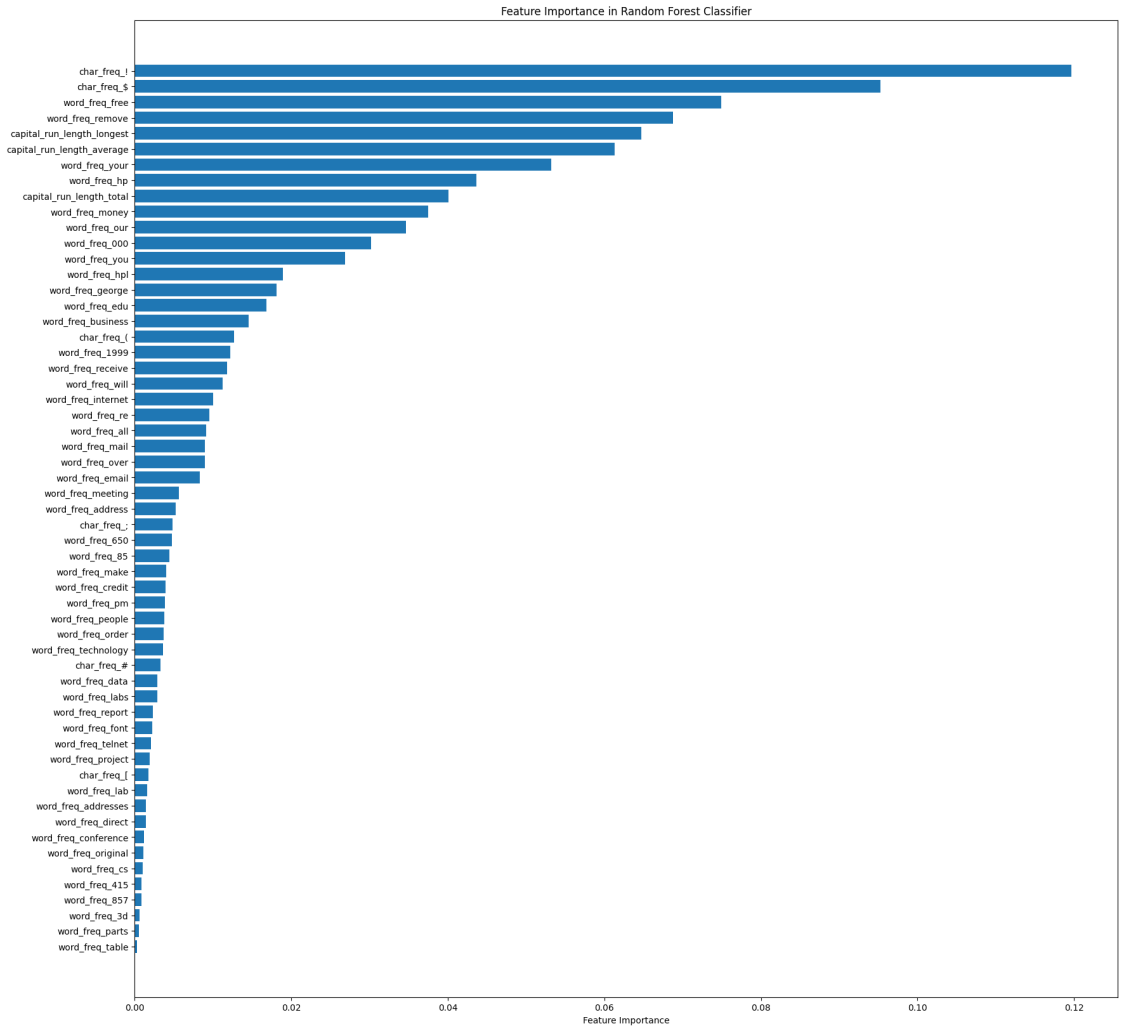


To determine the best set of predictors we can start by adding all predictors to a model and then testing to see which predictors are the most influences on the model by using the `feature_importances_` for random forests. Here is the first model with all the predictors.

[13]:
```
X = data[cols[:57]].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,␣
 ↪random_state=314)

rf = RandomForestClassifier(n_estimators=100, random_state=314)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
```

This graph shows what it looks like for the distribution of the feature importance. We can an exponential distribution and that the least influential features have very little influence on the model.

[14]:
```
importances = rf.feature_importances_
col_imp = pd.DataFrame([list(cols[:57]), list(importances)]).T.rename(columns =␣
 ↪{0:'feature', 1:'importance'}).sort_values('importance')

plt.figure(figsize = (20,20))
plt.barh(col_imp['feature'], col_imp['importance'])
plt.xlabel('Feature Importance')
plt.title('Feature Importance in Random Forest Classifier')
plt.show()
```



[15]:
```
# pd.DataFrame([list(cols), list(feature_importances)]).T.rename(columns = {0:
↪'feature', 1:'importance'})
```

[16]:
```
# cols[min(feature_importances) == feature_importances]
```

[17]:
```
# temp_cols = col_imp[:57]

# for i in range(2,57):
#     X = data[cols[:i]].values
#     acc, y_pred = randomForest(X,y)
#     acc_lst.append([i,acc*100])
```

[18]:
```
# temp_cols = cols[:57]
# temp_cols
```

Now we can iterate through and remove the worst predictor from the model each iteration and find

which the set of predictors produces the best model.

[19]:
```
i = 57
temp_cols = np.array(cols[:57])
acc_lst = []
col_lst = []

while len(temp_cols) > 2:
    X = data[temp_cols].values
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,␣
 ↪random_state=314)

    rf = RandomForestClassifier(n_estimators=100, random_state=314)
    rf.fit(X_train, y_train)
    y_pred = rf.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    pre = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1  = f1_score(y_test, y_pred)

    importances = rf.feature_importances_
    # print(importances)
    # print(min(importances))
    # print(min(importances) == importances)
    # print(temp_cols)
    # print(temp_cols[min(importances) == importances])
    removal = temp_cols[min(importances) == importances]
    temp_cols = temp_cols[min(importances) != importances]
    acc_lst.append([
        len(temp_cols),
        acc*100,
        pre*100,
        rec*100,
        f1*100,
        removal[0]])
    col_lst.append(temp_cols)
```

Here we can see the number of predictors in a given model, the worst predictor and the scores for the model.

[20]:
```
imp_acc = pd.DataFrame(acc_lst, columns = ['num_predictors', 'accuracy',␣
 ↪'precision', 'recall', 'f1', 'worst_predictor'])
imp_acc
```

[20]:
```
    num_predictors   accuracy  precision     recall         f1  \
0               56  95.482189  95.787140  92.903226  94.323144
1               55  95.134666  96.371882  91.397849  93.818985
```

2	54	94.874023	95.111111	92.043011	93.551913
3	53	95.308427	95.768374	92.473118	94.091904
4	52	95.047785	95.945946	91.612903	93.729373
5	51	94.960904	95.525727	91.827957	93.640351
6	50	94.787142	94.900222	92.043011	93.449782
7	49	94.787142	94.900222	92.043011	93.449782
8	48	95.308427	96.179775	92.043011	94.065934
9	47	95.047785	95.739910	91.827957	93.743139
10	46	94.874023	95.111111	92.043011	93.551913
11	45	95.134666	95.343681	92.473118	93.886463
12	44	94.700261	94.888889	91.827957	93.333333
13	43	95.134666	95.749441	92.043011	93.859649
14	42	95.134666	95.545657	92.258065	93.873085
15	41	94.700261	95.291480	91.397849	93.304061
16	40	94.960904	96.145125	91.182796	93.598234
17	39	94.787142	95.918367	90.967742	93.377483
18	38	95.134666	95.955056	91.827957	93.846154
19	37	94.960904	95.525727	91.827957	93.640351
20	36	94.700261	94.888889	91.827957	93.333333
21	35	94.787142	95.505618	91.397849	93.406593
22	34	94.265856	94.630872	90.967742	92.763158
23	33	94.787142	95.711061	91.182796	93.392070
24	32	94.526499	95.067265	91.182796	93.084523
25	31	94.874023	95.111111	92.043011	93.551913
26	30	94.787142	95.711061	91.182796	93.392070
27	29	95.134666	95.749441	92.043011	93.859649
28	28	94.787142	95.302013	91.612903	93.421053
29	27	94.874023	95.111111	92.043011	93.551913
30	26	94.787142	95.100223	91.827957	93.435449
31	25	94.526499	95.067265	91.182796	93.084523
32	24	94.526499	94.866071	91.397849	93.099671
33	23	94.265856	95.238095	90.322581	92.715232
34	22	94.439618	95.056180	90.967742	92.967033
35	21	94.265856	94.831461	90.752688	92.747253
36	20	94.352737	95.454545	90.322581	92.817680
37	19	94.265856	95.238095	90.322581	92.715232
38	18	94.265856	95.033860	90.537634	92.731278
39	17	94.005213	95.000000	89.892473	92.375691
40	16	94.178975	95.227273	90.107527	92.596685
41	15	94.439618	95.259594	90.752688	92.951542
42	14	94.613380	95.280899	91.182796	93.186813
43	13	94.092094	94.606742	90.537634	92.527473
44	12	93.136403	93.080357	89.677419	91.347207
45	11	93.570808	93.932584	89.892473	91.868132
46	10	93.397046	93.707865	89.677419	91.648352
47	9	93.049522	93.650794	88.817204	91.169978
48	8	93.049522	93.453725	89.032258	91.189427

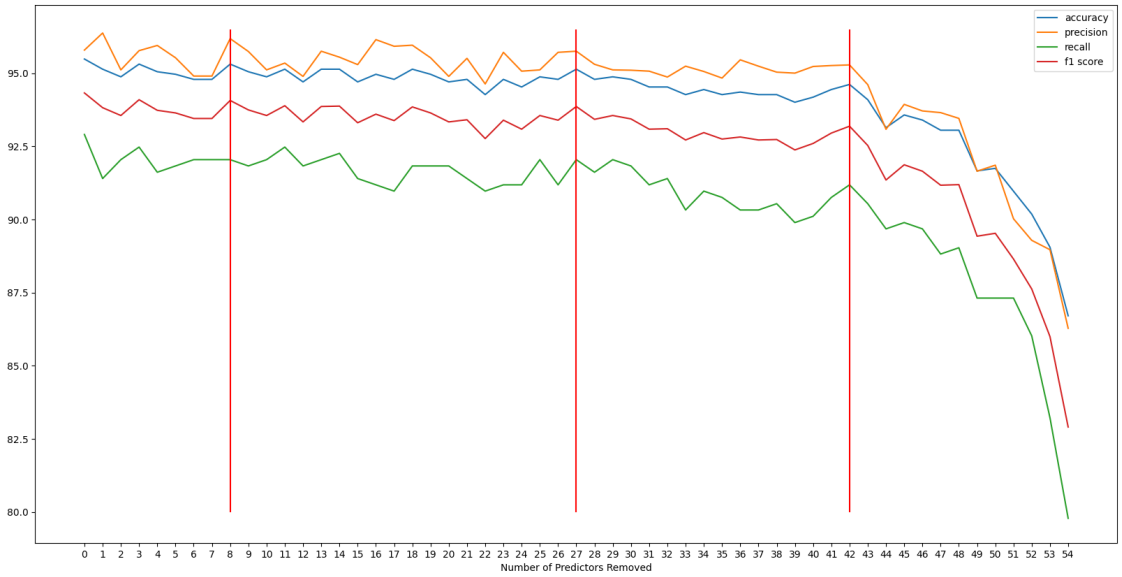
49	7	91.659427	91.647856	87.311828	89.427313
50	6	91.746308	91.855204	87.311828	89.525910
51	5	90.964379	90.022173	87.311828	88.646288
52	4	90.182450	89.285714	86.021505	87.623220
53	3	89.052997	88.965517	83.225806	86.000000
54	2	86.707211	86.279070	79.784946	82.905028

	worst_predictor
0	word_freq_table
1	word_freq_3d
2	word_freq_415
3	word_freq_parts
4	word_freq_cs
5	word_freq_857
6	word_freq_conference
7	word_freq_direct
8	word_freq_lab
9	word_freq_addresses
10	word_freq_original
11	word_freq_font
12	word_freq_project
13	word_freq_report
14	word_freq_telnet
15	char_freq_[
16	word_freq_85
17	word_freq_data
18	word_freq_pm
19	word_freq_credit
20	word_freq_labs
21	word_freq_people
22	word_freq_order
23	word_freq_make
24	char_freq_#
25	word_freq_address
26	char_freq_;
27	word_freq_technology
28	word_freq_meeting
29	word_freq_over
30	word_freq_650
31	word_freq_email
32	word_freq_mail
33	word_freq_receive
34	word_freq_re
35	word_freq_all
36	word_freq_business
37	word_freq_internet
38	word_freq_1999

```
39              word_freq_will
40               word_freq_hpl
41               word_freq_000
42            word_freq_george
43               word_freq_edu
44                 char_freq_(
45               word_freq_our
46             word_freq_money
47               word_freq_you
48                word_freq_hp
49  capital_run_length_longest
50    capital_run_length_total
51              word_freq_your
52              word_freq_free
53            word_freq_remove
54                 char_freq_$
```

After graphing the scores for each model we can see that after removing 42 predictors we have the best set of scores for the least number of predictors (14).

[21]:
```
plt.figure(figsize = (20,10))
plt.plot(56-imp_acc['num_predictors'], imp_acc['accuracy'], label = 'accuracy')
plt.plot(56-imp_acc['num_predictors'], imp_acc['precision'], label =␣
 ↪'precision')
plt.plot(56-imp_acc['num_predictors'], imp_acc['recall'], label = 'recall')
plt.plot(56-imp_acc['num_predictors'], imp_acc['f1'], label = 'f1 score')

plt.title('' , size = 15)
plt.xlabel('Number of Predictors Removed', size = 10)
plt.ylabel('', size = 10)
plt.legend()

plt.xticks(np.arange(0, 55, 1))

plt.vlines(x=[8,27,42], ymin = 80, ymax = 96.5, color = 'red')
plt.show()
```



With an accuracy score of 94% we can see that a trimmed random forest model produces an extremely accurate spam filter with only 14 predictors. Using more complex methods could definitely improve this value but overall it performs well enough to accurately identify a vast majority of emails.

[22]:
```
X = data[col_lst[42]].values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,␣
 ↪random_state=314)

rf = RandomForestClassifier(n_estimators=100, random_state=314)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)

acc = accuracy_score(y_test, y_pred)*100
pre = precision_score(y_test, y_pred)*100
rec = recall_score(y_test, y_pred)*100
f1  = f1_score(y_test, y_pred)*100

print('14 predictor trimmed random forest model:')
print('Accuracy score:  {:.2f}%'.format(acc))
print('Precision score: {:.2f}%'.format(pre))
print('Recall score:    {:.2f}%'.format(rec))
print('F1 score:        {:.2f}%'.format(f1))
```

```
14 predictor trimmed random forest model:
Accuracy score:  94.09%
Precision score: 94.61%
Recall score:    90.54%
F1 score:        92.53%
```